

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет
Лабораторная работа №2

Выполнил:
Шайкин Андрей
Группа
J3210

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Целью данной лабораторной работы является реализация взаимодействия клиентской части приложения с внешним API. Необходимо привязать разработанную ранее вёрстку к серверным данным, используя методы *fetch*, *axios* или *XHR*.

Основные требования:

- Реализация мокового API средствами **json-server**.
- Подключение авторизации (регистрация и вход) с использованием **json-server-auth**.
- Работа с защищенными роутами через передачу токена в заголовках запроса (**Bearer Token**).
- Интеграция реального внешнего API для получения динамических данных.
- Реализация динамического рендеринга контента на основе данных, полученных с сервера.

Ход работы

На первом этапе была подготовлена серверная часть приложения. С помощью инструмента **json-server** был поднят локальный сервер на порту 3000. Создана база данных в формате JSON (*db.json*), содержащая массивы пользователей и объектов курсов с их характеристиками: названием, описанием, ценой и категорией.

```
Шайкин_Андрей > labs > lab2 > dbjson > {} 0
{
  "users": [
    {
      "email": "bebra@gmail.com",
      "password": "$2a$10$dbaMcY8.1TvkHvqLstNED..cn0vou3UR0mm5jG3EPQ61dkOoFkMO6",
      "firstName": "Иван",
      "lastName": "Иванов",
      "id": 1
    }
  ],
  "courses": [
    {
      "id": 1,
      "title": "Vue.js 3 [ ] нуля (Продвинутый курс)",
      "category": "Frontend",
      "level": "Junior",
      "price": 15000,
      "desc": "Создаем SPA приложения. Composition API, Vuex, Vue Router.",
      "image": "img/course1.png",
      "badge": "bg-primary text-info border-info"
    },
    {
      "id": 2,
      "title": "Python Django",
      "category": "Backend",
      "level": "Middle",
      "price": 35000,
      "desc": "Разработка надежных бекенд систем и REST API.",
      "image": "img/course2.png",
      "badge": "bg-success text-success border-success"
    }
  ]
}
```

Рис. 1: Структура базы данных в файле db.json

Далее была реализована логика аутентификации. С помощью метода **POST** данные из форм регистрации и входа отправляются на сервер. При успешной авторизации сервер возвращает JWT-токен, который сохраняется в **localStorage** браузера. Настроена защита страниц: при отсутствии токена доступ к личному кабинету блокируется, и пользователь перенаправляется на страницу авторизации.

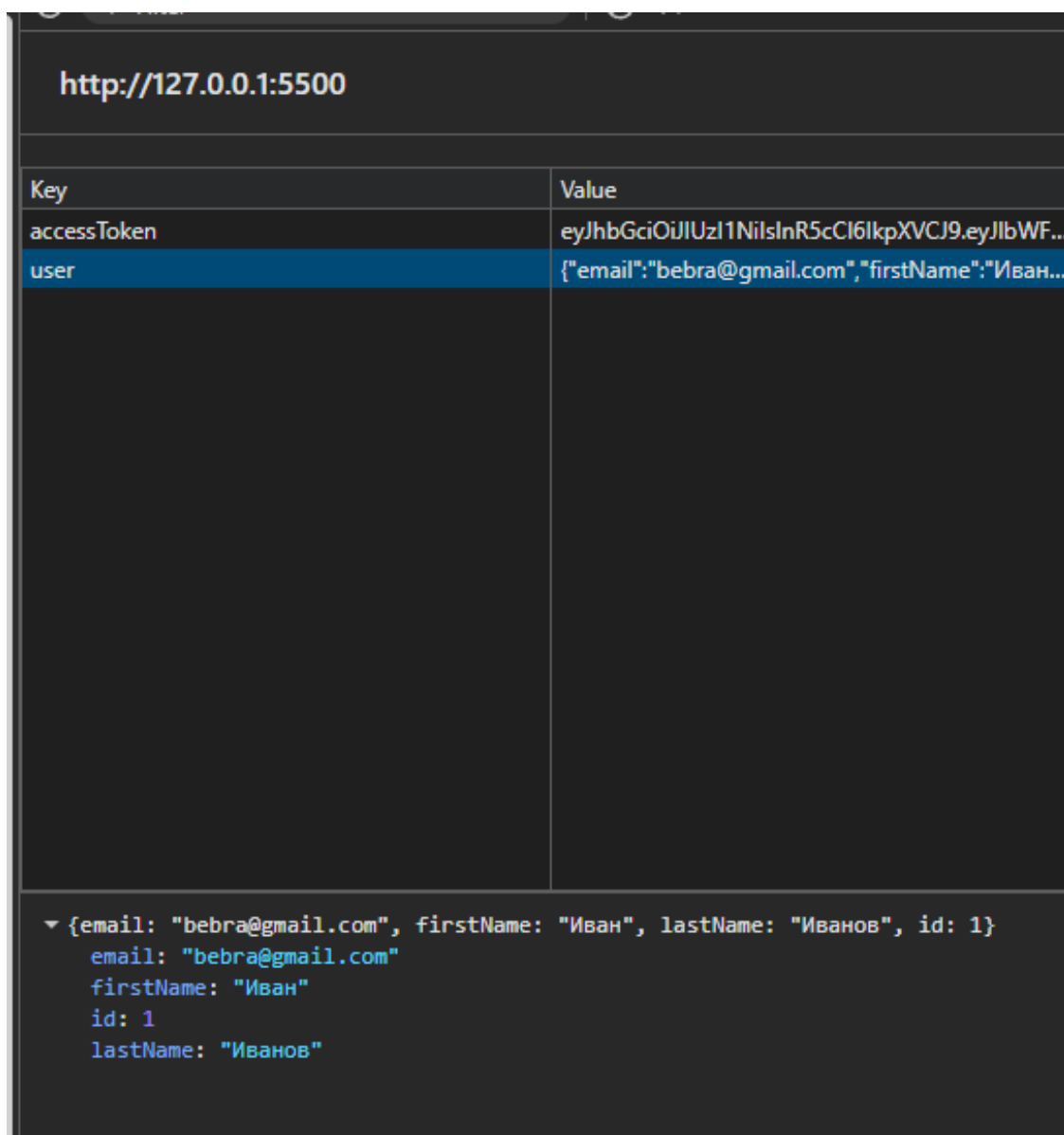


Рис. 2: Сохранение токена доступа в Local Storage

Для реализации требований по взаимодействию с данными была переписана логика каталога курсов. Теперь карточки не прописаны в HTML-коде, а генерируются динамически через JavaScript после получения данных от API. Запросы к серверу включают заголовок **Authorization** с типом *Bearer*, что подтверждает права доступа пользователя.

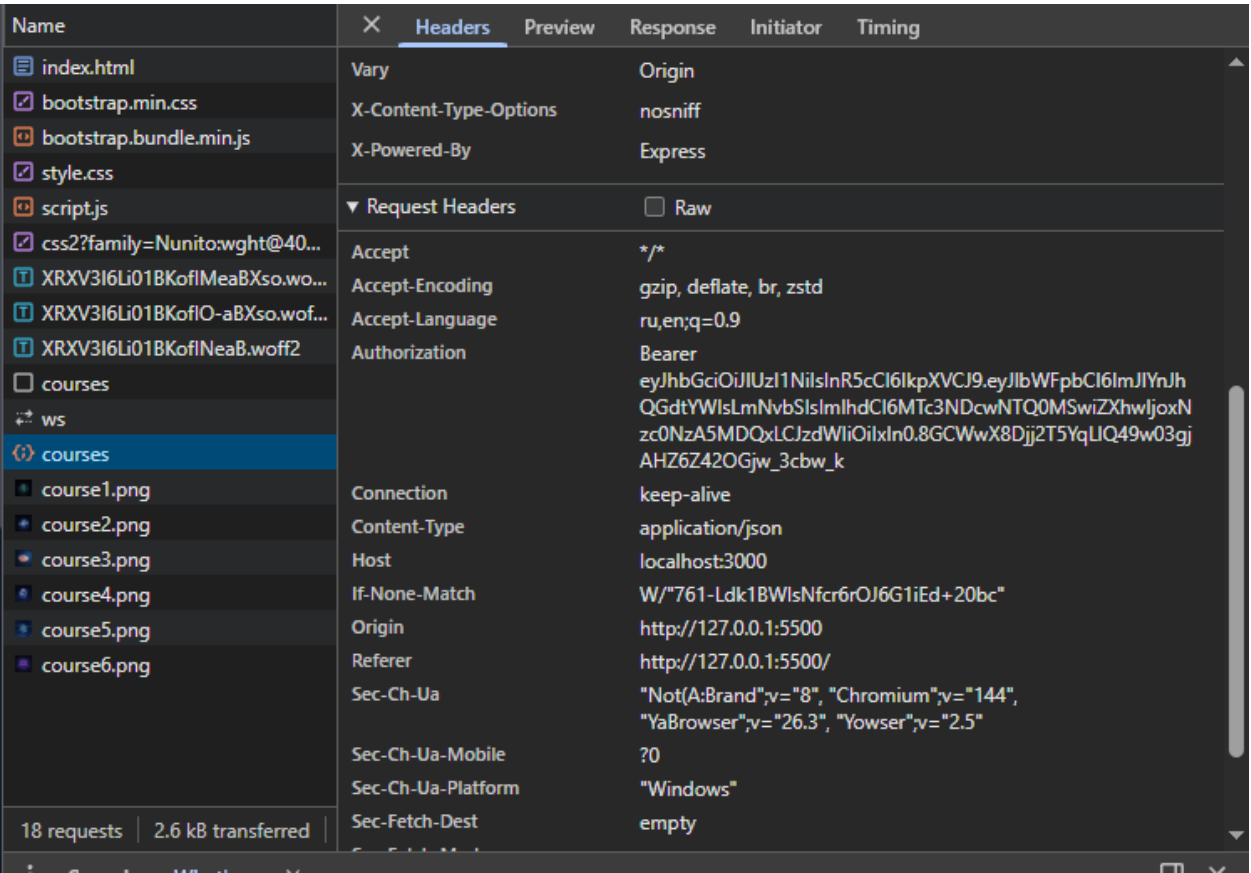


Рис. 3: Передача Bearer токена в заголовках сетевого запроса

В соответствии с заданием, в проект была интегрирована работа с **внешним API**. В личном кабинете реализован запрос к сервису *AdviceSlip API*, который при каждой загрузке страницы возвращает случайную цитату (совет дня). Это демонстрирует умение работать с асинхронными запросами к сторонним ресурсам.

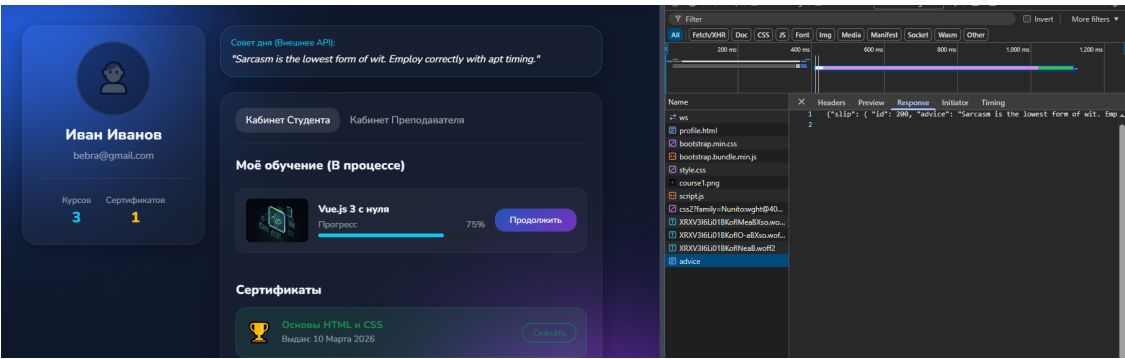


Рис. 4: Блок данных, полученных из внешнего стороннего API

Вывод

В ходе выполнения лабораторной работы №2 я закрепил навыки работы с асинхронным JavaScript и протоколом HTTP.

Я освоил процесс создания моковых серверов для имитации бэкенда, научился работать с библиотекой *json-server-auth* для реализации полноценной системы регистрации и входа. В процессе разработки были изучены механизмы работы с **Promises** и синтаксисом **async/await**. Я научился собирать данные форм через объект *FormData*, отправлять их на сервер и обрабатывать JSON-ответы.

Особое внимание было уделено безопасности: реализована проверка наличия токена доступа и его передача в защищенные роуты. Также был получен практический опыт интеграции сторонних публичных API в интерфейс приложения. Разработанная система фильтрации в каталоге теперь работает динамически, обрабатывая массив данных в реальном времени. Таким образом, проект превратился из статического макета в функциональное веб-приложение, готовое к дальнейшей миграции на современные фреймворки.